20: MultiThreading I

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Zoom

- Lecture using Zoom:
  - https://iu.zoom.us/j/362983973
  - Password: ENGR315

- Office Hours:
  - Wed 5:30 PM - 8:00 PM, Th 2:30 PM - 4:00 PM
  - https://iu.zoom.us/j/770949556

P4: Bitcounting

- How did that go?

PX is out!

Project X: Accelerating Machine Learning

E315, Spring 2020

Version 2020.0

Preliminary Report Due: 11:59pm, April 27th, 2020

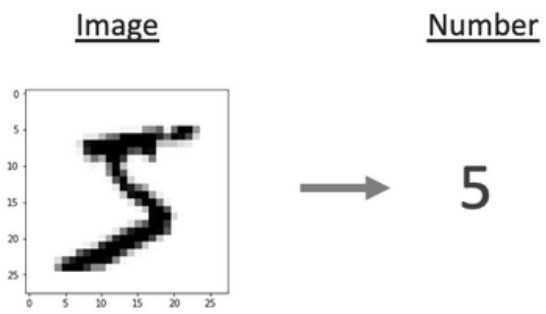
Demo and Final Report Due: 2.30pm May 8th, 2020

Goal

The goal of this project is to accelerate a reference hand-written digit classification machine learning implementation. You will then write a report to document your design.

PX Highlights

- Goal: Accelerate Digit Classification



pre-trained

- Python reference implementation given

PX: Your task

==========

This is the part you need to accelerate:

==========

```
start = timeit.default_timer()

corrects, wrongs = evaluate(net, test_imgs, test_labels)

stop = timeit.default_timer()

print ("Total Correct:" + str(corrects))
print ("Total Incorrect: " + str(wrongs))
print("Overall Accruracy: " + str(corrects / ( corrects + wrongs)))
print("Overall Accruracy (%): " + str( int( 1000* corrects / ( corrects + wrongs)) / 10) + "%")
print ()
print('Run Time: ' + str(stop - start) + ' Seconds')
```

```
Total Correct:9565
Total Incorrect: 434
Overall Accruracy: 0.9565956595659566
Overall Accruracy (%): 95.6%

Run Time: 22.37916092100204 Seconds
```

You CAN

- Work in groups of 2
- Use FPGA (or Not)
- Use Multithreading / MultiProcessing 2 CPUs
- Tweak ML algorithm

* Python, C, C++, Haskell, Lisp, etc...

You CANNOT

- Change testing / training dataset
- Receive credit for accelerating using numpy’s tanh.
  - Too easy.

Grading

- Final Project grade determined by application speedup and accuracy
  - 25% of semester grade

- Final Exam grade determined by Reports.
  - Preliminary Report
  - Final Report

  - 20% of semester grade

Preliminary Report

- Who's on your team
- What you are currently working on
- What you are planning on trying.
- Why do you think what you are doing will work?

- 1 page
- Must include at least 1 figure / diagram

Final Report

- Who's on your team
- What you did:
  - What worked
  - What didn't work
- How does it work?
  - What are your final performance numbers?
  - Why is that reasonable?
- Who did what
- ~~Analsis~~ Analysis

Final Report Analysis

- What did you *learn?*
- What did you expect to work that didn't? Why?
- What was interesting or non-intuitive?
- Did I give you any bad advice?
- If you could start over again, what would you try?

~5 pages

4.5 pages

five pages

Multithreading (Review from E201)

- What's the difference between Process and Thread?

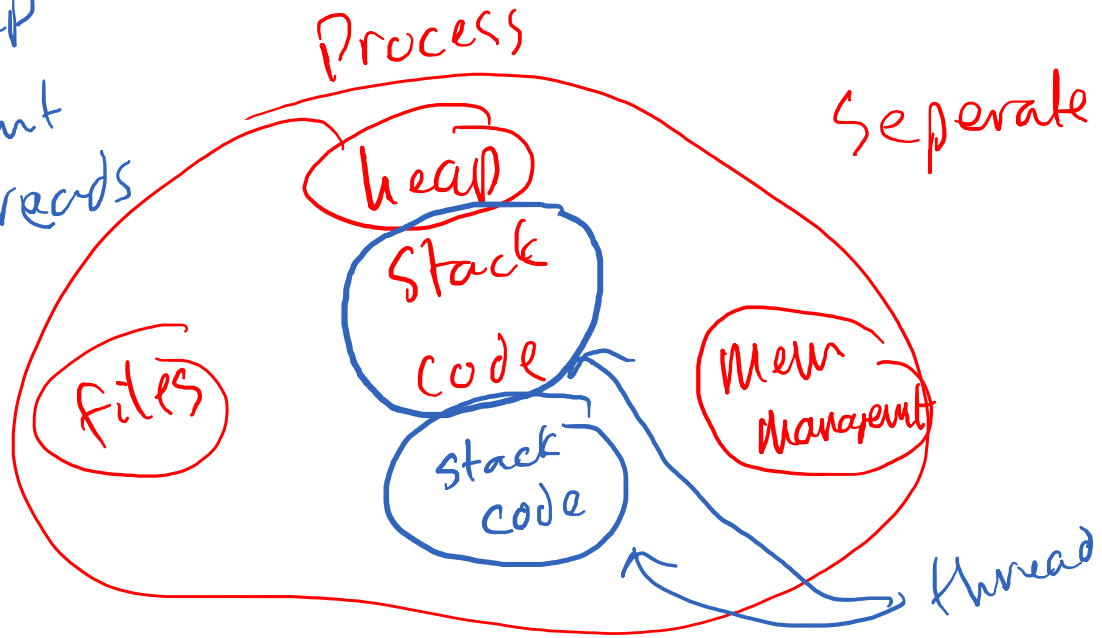
same heap

separate stacks

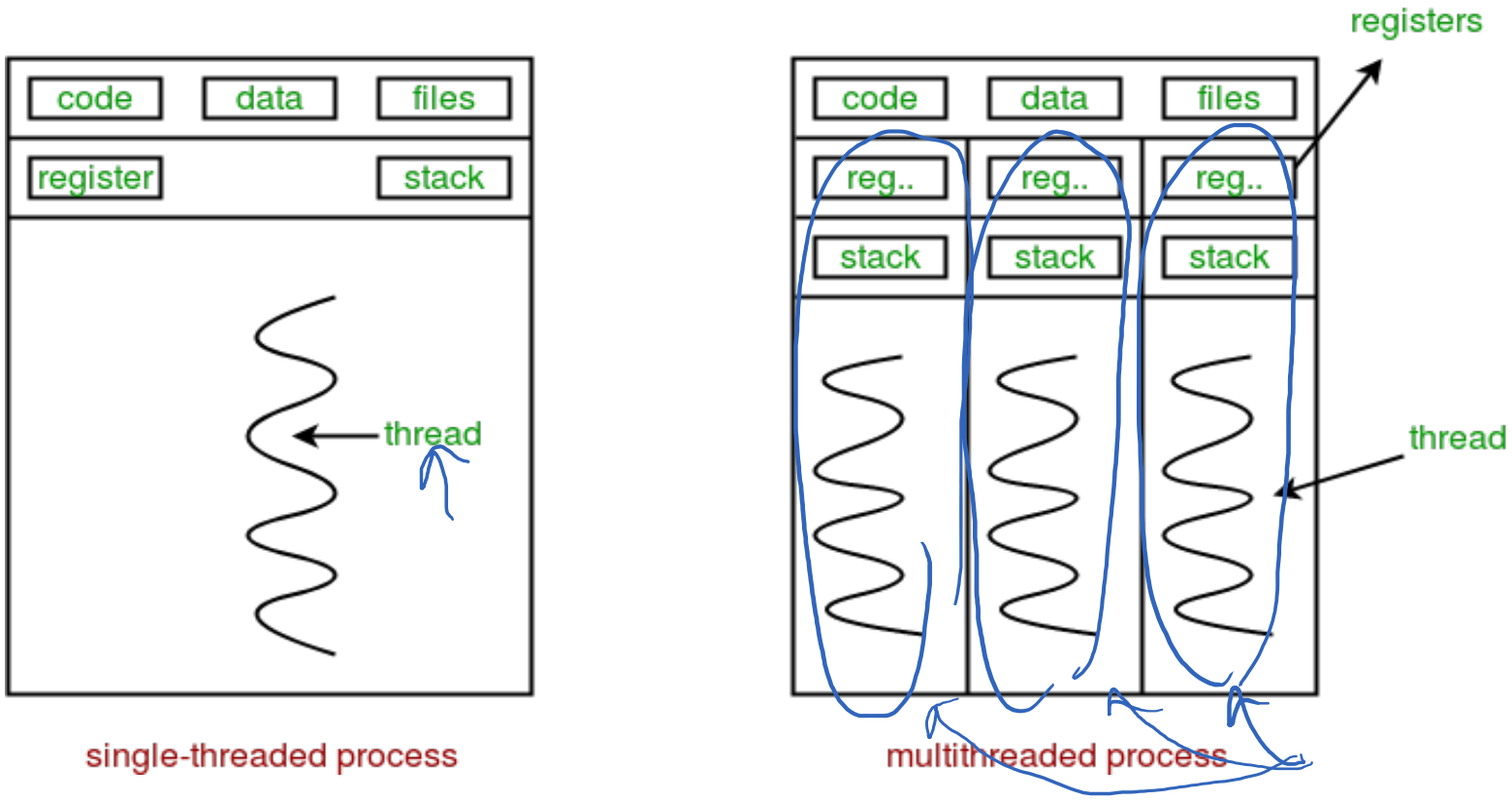
executing process

different heap

different threads

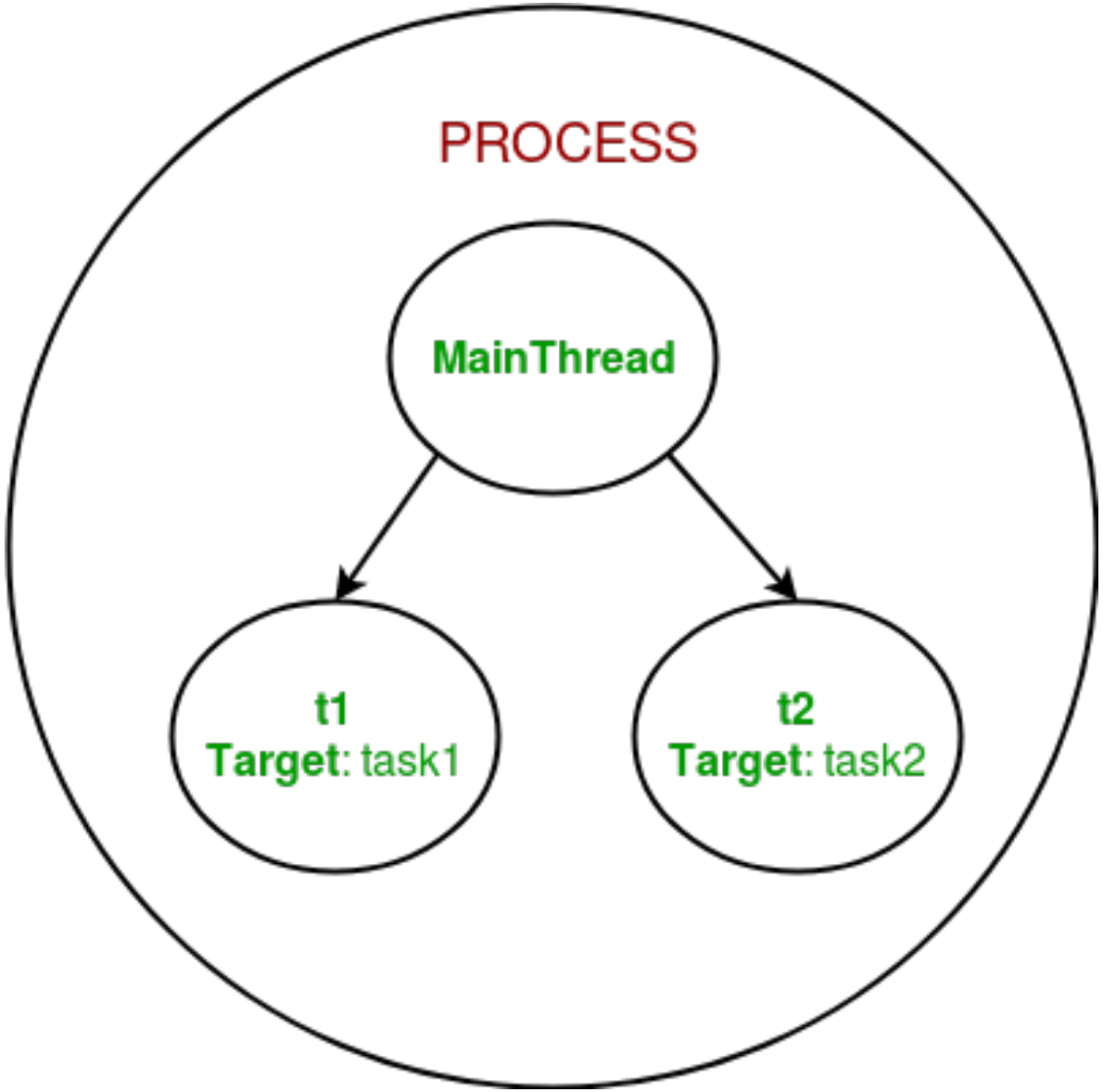


MultiThreading



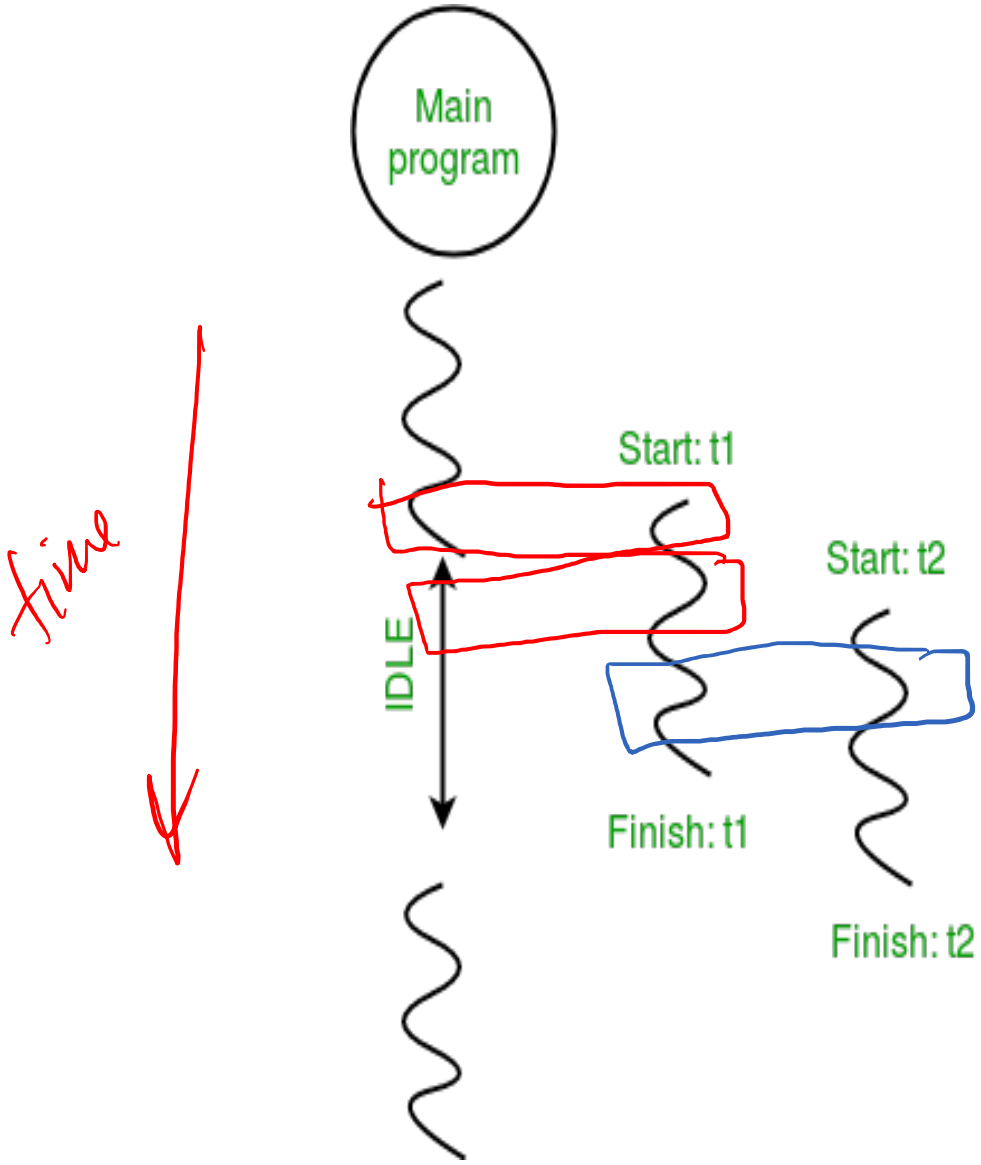
https://www.geeksforgeeks.org/multithreading-python-set-1/

MultiThreading



https://www.geeksforgeeks.org/multithreading-python-set-1/

MultiThreading



https://www.geeksforgeeks.org/multithreading-python-set-1/

Multithreading in Python

```
import threading
def foo(x,y):
    x = x + y

t = Threading.Thread(target=foo, args=(arg0, arg1))
t.start()
t.join()
```

start thread

block until thread t finishes

Multithreading +Queuing in Python

```
import threading
import queue
def foo(x,y,q):
     q.put(x + y)

q = queue.Queue()
t = Threading.Thread(target=foo, args=(arg0, arg1, q))
t.start()
t.join()
x_plus_y = q.get()
```

Searching an unordered array

```
import numpy as np
def find(needle, haystack):
    return np.where( haystack == needle)[0]
```

split it

```
buf = np.random.random(1000000)
idx = find( 0xcd67cc13, buf)
```

Searching an unordered array

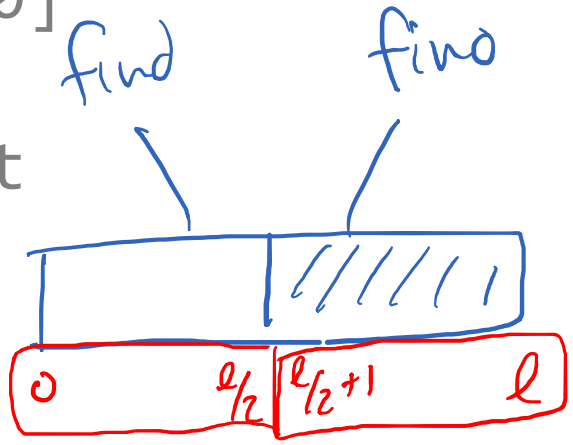
```
import numpy as np
def find(needle, haystack):
    return np.where( haystack == needle)[0]

buf = np.random.random(1000000)
idxs = find( 0xcd67cc13, buf)
```

Searching an unordered array

```
def find(needle, haystack):
    return np.where( haystack == needle)[0]
def find2(needle, haystack, offset):
    return find(needle, haystack) + offset

buf = np.random.random(1000000)
half_sz = int(len(buf)/2)
idx1 = find2(needle, buf[:half_sz], 0)
idx2 = find2(needle, buf[half_sz:], half_sz)
idxs = np.concatenate((idx1,idx2), axis=0)
```



find (0, l/2)

l/2 + find (0, l/2)

Searching an unordered array

```
def find(needle, haystack):
    return np.where( haystack == needle)[0]
def find2(needle, haystack, offset):
    return find(needle, haystack) + offset
def find3(needle, haystack, offset, q):
    q.put(find2(needle,haystack, offset))
```

(large chin)

needle = 0x e0 67 cc 13

```
buf = np.random.random(1000000)
half_sz = int(len(buf)/2)
q = queue.Queue()
t1 = threading.Thread(target=find3, args=(needle,
buf[:half_sz], 0, q))
t2 = threading.Thread(target=find3, args=(needle,
buf[half_sz:], half_sz, q))
t1.start(); t2.start()
t1.join(); t2.join()
idxs = np.concatenate((q.get(), q.get()), axis=0)
```

line wrap

[1, 2, 3] → [3, 1, 2]

Runtimes on Pynq

- Single-Threaded: 0.4591 Seconds
- Multi-Threaded: 0.2438 Seconds
- Yay, Multithreading works!

Bitcount: How to multithread?

```
def bitcount(num):
    return bin(num).count('1')
W = np.random.random(1000000)
sum_1s = sum(map(bitcount,W))
```

w = [1, 2, 3]

x = map(bitcount, w)

x = [1, 1, 2]

sum(map(bitcount, w))

= sum[1, 1, 2]

= 4

w = [1, 2, 3]

map(bitcount, w) ⇒

= [bitcount(1), bitcount(2), bitcount(3)]

= [1, 1, 2]

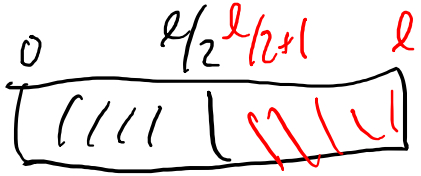
z = []

for i in range(len(w)):

z.append = bitcount(w[i])

Bitcount: How to multithread?

```
def bitcount(num):
    return bin(num).count('1')
W = np.random.random(1000000)
half_sz = int(len(W)/2)
cnt1 = sum(map(bitcount, W[:half_sz]))
cnt2 = sum(map(bitcount, W[half_sz:]))
sum_1s = cnt1 + cnt2
```



Bitcount: How to multithread?

```
def bitcount(num):
    return bin(num).count('1')
def bitcount_mt(v, q):
    q.put(sum(map(bitcount,v)))
```

```
W = np.random.random(1000000)
q = queue.Queue()
half_sz = int(len(W)/2)
t1 = threading.Thread(target=bitcount_mt, args=(W[:half_sz], q))
t2 = threading.Thread(target=bitcount_mt, args=(W[half_sz:], q))
t1.start()
t2.start()
t1.join()
t2.join()
sum_1s = q.get() + q.get()
```

Bitcount on Pynq

- Single-Threaded: 22.1146 Seconds
- Multi-Threaded: 22.0863 Sec

Bitcount on Pynq

- Single-Threaded: 22.1146 Seconds
- Multi-Threaded: 22.0863 Seconds
- What happened?
  - Global Interpreter Lock (GIL)

Computing a Dot Product

```
W = np.array([[1,2,3],[4,5,6]])
X = np.array([7,8,9])
Y = np.dot(W,X)
```

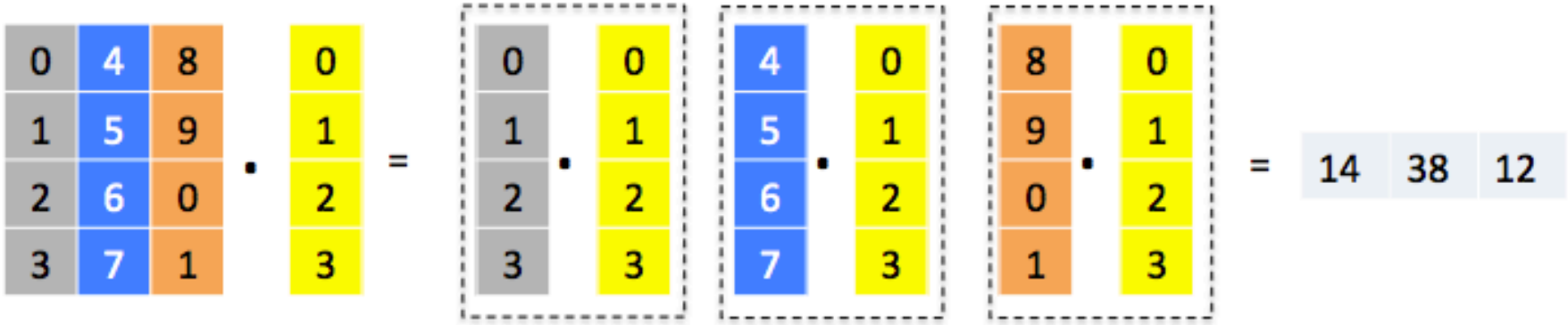
Weight

img

$$\begin{bmatrix} 1, 2, 3 \\ 4, 5, 6 \end{bmatrix} \cdot \begin{bmatrix} 7 \\ 8 \\ 9 \end{bmatrix} = \begin{bmatrix} \overbrace{1 \cdot 7 + 2 \cdot 8 + 3 \cdot 9}^{\text{th 1}}, & \overbrace{4 \cdot 7 + 5 \cdot 8 + 6 \cdot 9}^{\text{thread 2}} \end{bmatrix}$$

$$\begin{bmatrix} 50, & 122 \end{bmatrix}$$

Computing a Dot Product



A dot product between a matrix and a vector…

…is evaluated by taking dot products between the columns of the matrix and the vector

http://arrayfire.org/docs/matrix_vector_dot_product.png

Dot Product + Tanh

thread 0 | thread 1

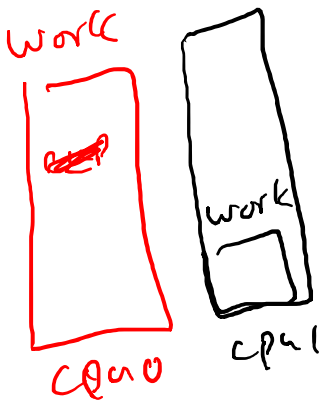
$\begin{bmatrix} .1 & .2 & .3 \\ .4 & .5 & .6 \end{bmatrix} \begin{bmatrix} .7 \\ .8 \\ .9 \end{bmatrix}$ | tanh(...)

option 2

option 1

thread 0: $\tanh\left(\begin{bmatrix} .1 & .2 & .3 \end{bmatrix} \begin{bmatrix} .7 \\ .8 \\ .9 \end{bmatrix}\right)$

thread 1: $\tanh\left(\begin{bmatrix} .4 & .5 & .6 \end{bmatrix} \begin{bmatrix} .7 \\ .8 \\ .9 \end{bmatrix}\right)$




```
W = np.array([[.1,.2,.3],[.4,.5,.6]])
X = np.array([.7,.8,.9])
Y = np.dot(W,X)
Z = np.tanh(Y)
```

more even?

$\begin{bmatrix} .1 & .2 & .3 \\ .4 & .5 & .6 \end{bmatrix} \begin{bmatrix} .7 \\ .8 \\ .9 \end{bmatrix} = \begin{bmatrix} 0.5 & 1.22 \end{bmatrix} \Rightarrow \tanh(\begin{bmatrix} 0.5, & 1.22 \end{bmatrix}) =$

$\Rightarrow \begin{bmatrix} 0.46, & 0.83 \end{bmatrix}$

Repeated Weights

```
Ws =[ np.array([[.1,.2,.3],[.4,.5,.6]]),
      np.array([[.1,.2],[.3,.4],[.5,.6]]) ]
X = np.array([.7,.8,.9])
for W in Ws:
    Y = np.dot(W,X)
    Z = np.tanh(Y)
    X = Z
```

Repeated Weights + Inputs

```
Ws =[ np.array([[.1,.2,.3],[.4,.5,.6]]),
      np.array([[.1,.2],[.3,.4],[.5,.6]]) ]
Xs = [ np.array([.7,.8,.9]),
        np.array([.1, .2, .3])]
Zs = []
for X in Xs:
    for W in Ws:
        Y = np.dot(W,X)
        Z = np.tanh(Y)
        X = Z
      Zs.append(Z)
```

20: MultiThreading I

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University